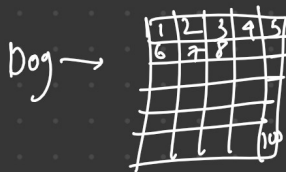
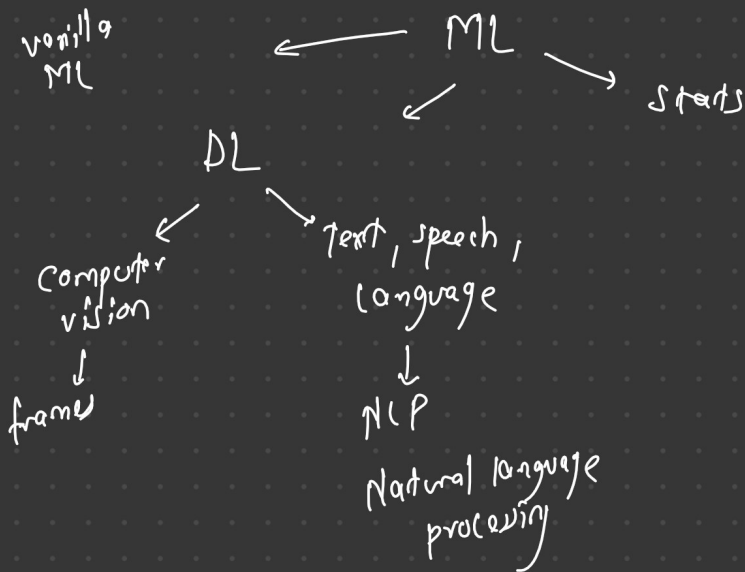


Agenda — 09-02-2025



↓ CNN

Hi, How are you? → ascii
0 1 2 3 4 → unicode

1 0 2 4 10
→ expect

4 → you

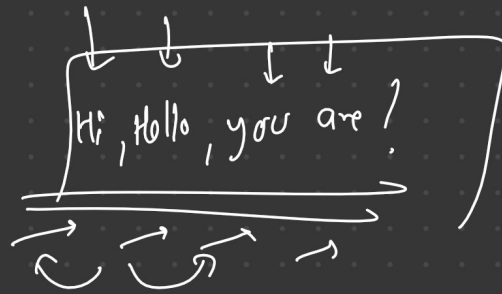
Hi → 0 0
How → 0 0 0
are → 0 0 0 0 → 0/1
you → 0 0 0
? → 0 0

→ ANN

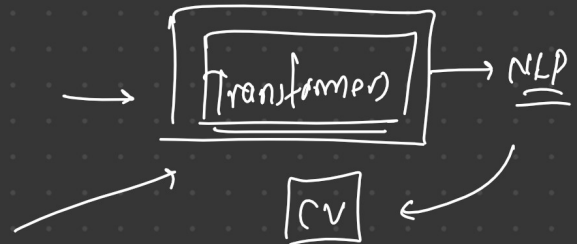
Hi, How have you been?
0 → 5

ANN → fixed input
CNN → extract meaningful pixels in terms of vision.

NLP: —



~~Image~~



Encoding: —

Dog, cat, person → Actual y

→ label encoding.

Dog → 1
cat → 2
person → 3

Hi, How are you?
↓ ↓ ↓ ↓ ↓
0 1 2 3 4 5

Dictionary → 100,000

→ 0 → 99,999

→ Too many labels

→ you > hi

5 > 0



One-hot-encoding

Dog
cat
person

v_1	v_2	v_3	
1	0	0	$\rightarrow L_1$
0	1	0	$\rightarrow L_2$
0	0	1	$\rightarrow L_3$

$$L_1 > L_3 \rightarrow \text{NO}$$

$\rightarrow$ sparse matrix, High dimensional matrix.

$$w_1 \rightarrow \begin{matrix} 100,000 \\ \left[\begin{array}{cccc} - & - & - & - \end{array} \right] \\ 1 \times 100,000 \end{matrix}$$

$$\boxed{100,000 \times 100,000}$$

$\boxed{\text{word}}$

$\rightarrow U_n$

Count

$\boxed{200}$

$$\boxed{200 \times 200}$$

TF-IDF

Term Frequency - Inverse Document Frequency

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$
$$\text{TF} \times \log\left(\frac{N}{\text{DF}}\right)$$

TF $\rightarrow$ How often a word appears in a document.

N $\rightarrow$ Total no. of documents.

DF $\rightarrow$ No of document contain the word

IDF $\rightarrow$ How unique a word is across all document.

Doc 1: "apple banana cherry apple"

Doc 2: "cherry date cherry banana"

Doc 3: "date apple date date"

TF : Doc 1 $\rightarrow$ (Total words $\rightarrow$ 4)

apple: 2 $2/4 = 0.5$

banana: 1 $1/4 = 0.25$

cherry: 1 $1/4 = 0.25$

IDF:

word	Appears in How many Doc	IDF ($\log(N/df)$)
apple	2	$\log(3/2) \approx 0.18$
banana	2	$\log(3/2) \approx 0.18$
cherry	2	$\log(3/2) \approx 0.18$
date	2	$\log(3/2) \approx 0.18$

Compute - TF-IDF

Apple in Doc 1 : $TF * IDF = 0.5 * 0.18 \rightarrow 0.09$

0, 0, 0, 0, 0, rare $\rightarrow$

TF, IDF, Apple $\rightarrow$ important

word 2 vec

→ neural network

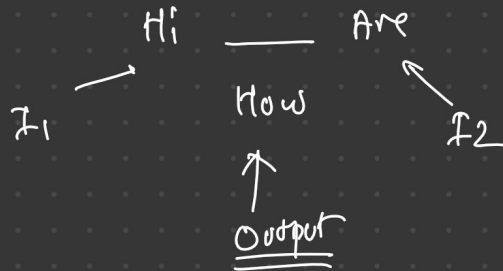
→ C Bow (continuous Bag of words)

predict a word based on surrounding word

→ skip-gram:

predict surrounding word given a word.

C Bow →



window size = 2

"The cat sits on the mat"

Training data:

The	sits	cat
cat	on	sits
sits	the	on
on	mat	the

Step 1: —

create vocab:

"The cat sits on mat"

① ② ③ ④ ⑤

ONE	①	②	③	④	⑤
The	1	0	0	0	0
cat	0	1	0	0	0
sits	0	0	1	0	0
on	0	0	0	1	0
Mat	0	0	0	0	1

→ Step 2:

Initialize random embeddings

→ 3

$$\begin{array}{l}
 \text{The} \\
 \text{cat} \\
 \text{sits} \\
 \text{on} \\
 \text{mat}
 \end{array}
 \begin{bmatrix}
 0.2 & 0.4 & 0.6 \\
 0.1 & 0.3 & 0.5 \\
 0.4 & 0.2 & 0.1 \\
 0.7 & 0.5 & 0.3 \\
 0.1 & 0.2 & 0.3
 \end{bmatrix}
 \rightarrow
 \begin{bmatrix}
 0.2 & 0.4 & 0.6 \\
 0.1 & 0.3 & 0.5 \\
 0.4 & 0.2 & 0.1 \\
 0.7 & 0.5 & 0.3 \\
 0.1 & 0.2 & 0.3
 \end{bmatrix}$$

Embedding matrix 5×3

$$\begin{array}{l}
 \text{apple} \quad 0.2 \quad 0.4 \quad 0.6 \rightarrow \\
 \text{green apple} \quad 0.19 \quad 0.38 \quad 0.65 \rightarrow
 \end{array}
 \rightarrow \text{expected output}$$

→ Example sets.

$$\begin{array}{l}
 \text{"The"} \rightarrow \text{OHE} \rightarrow \text{embedding matrix} \rightarrow [1, 0, 0, 0, 0] \\
 \downarrow \text{index} \quad \nearrow \\
 [0.2, 0.4, 0.6]
 \end{array}$$

$$\text{"sits"} \rightarrow [0, 1, 0, 0, 0] \rightarrow [0.4, 0.2, 0.1]$$

→ Hidden layer representation

$$\begin{aligned}
 h &= \frac{1}{2} (e_{\text{the}} + e_{\text{sits}}) \\
 &= \frac{1}{2} \left([0.2, 0.4, 0.6] + [0.4, 0.2, 0.1] \right) \\
 h &= [0.15, 0.35, 0.55]
 \end{aligned}$$

unique word

unique word

unique word

unique word

unique word

x_1^1 x_2^1 x_3^1

x_1^2 x_2^2 x_3^2

x_1^3 x_2^3 x_3^3

x_1^4 x_2^4 x_3^4

x_1^5 x_2^5 x_3^5

$$\rightarrow \text{HL-output} = \frac{\text{Emb-1} + \text{Emb-2}}{2}$$

$\uparrow \uparrow \uparrow$
w matrix $\rightarrow v \times N$

w^l (learnable matrix) $\rightarrow N \times V \rightarrow$ vocal size
 $\downarrow$
Embedding dimension

$$y = h \cdot w^l$$

$$h = [0.3 \ 0.3 \ 0.35]$$

$$y_1 = [0.3 \ 0.3 \ 0.35]$$

$$y_4 = [0.3 \ 0.3 \ 0.35]$$

0.3	0.5	0.7	0.2	0.1
0.6	0.4	0.2	0.8	0.3
0.9	0.7	0.5	0.4	0.2

3x5

$\downarrow \downarrow$	$\downarrow \downarrow$	$\downarrow \downarrow$	$\downarrow \downarrow$	$\downarrow$
0.3	0.5	0.7	0.2	0.1
0.6	0.4	0.2	0.8	0.3
0.9	0.7	0.5	0.4	0.2

$\uparrow \uparrow \uparrow \uparrow \uparrow$

the

$$h = [0.3, 0.3, 0.35]$$

$$y^1 = 0.585$$

$$y^2 = 0.515$$

$$y_3 = 0.445$$

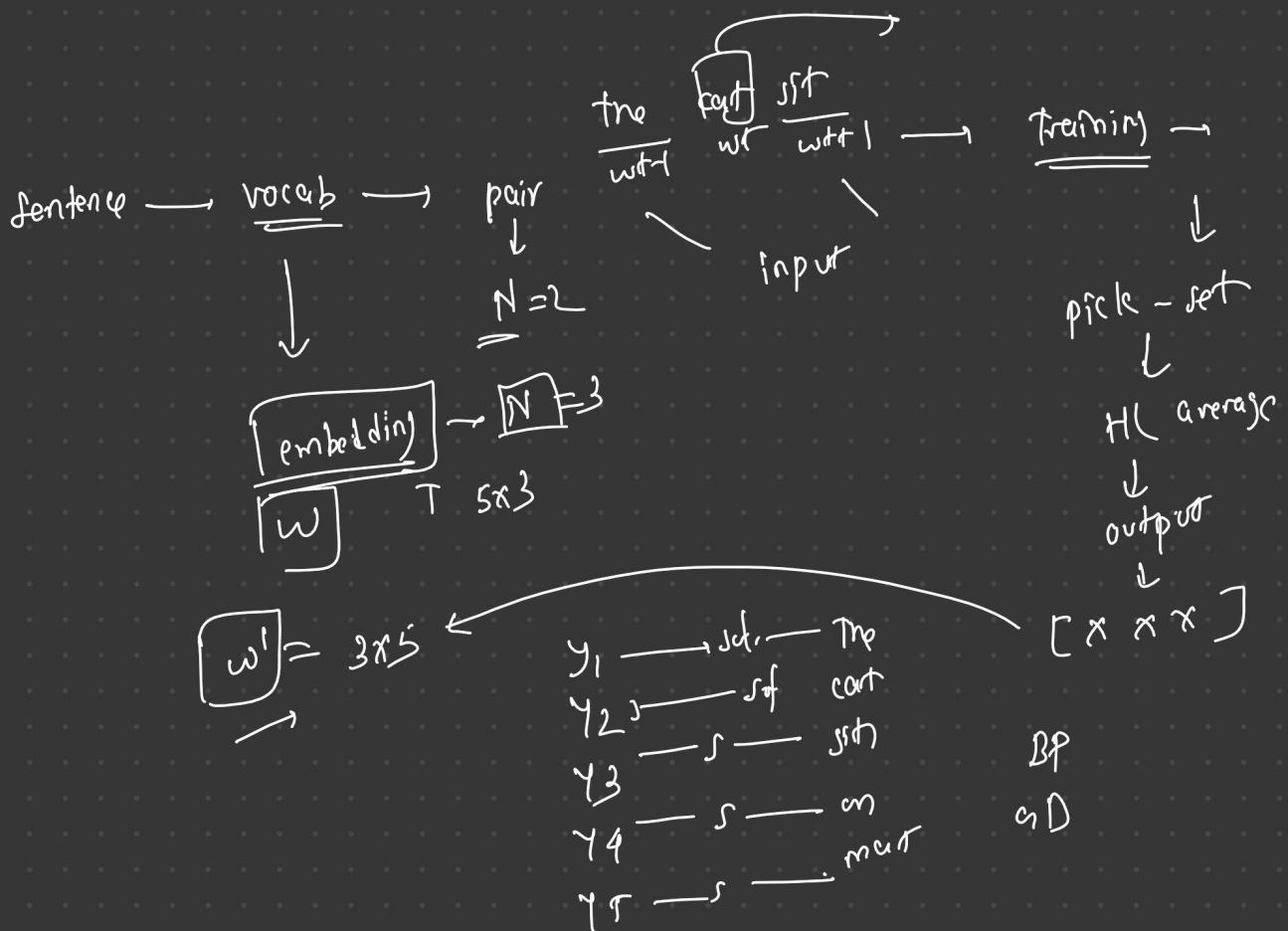
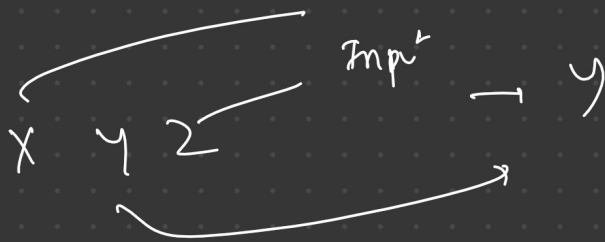
$$y_4 = 0.44$$

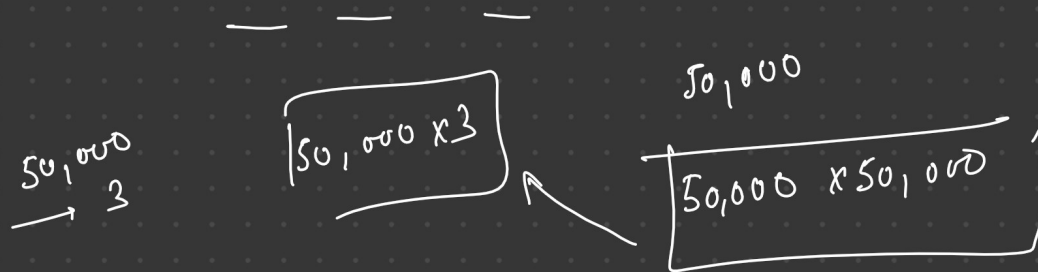
$$y_5 = 0.19$$

raw o/p $\rightarrow$ softmax $\rightarrow$

The	0.1	0
cart	0.8	1
sft	0.3	0
on	0.4	0
met	0.2	0

BP, GD





Skip-grams

predict context given a word

"The cat sat on the mat"

cat The sat
sat cat on
the one mat
on sat the

"cat" $\rightarrow$ embedding $\rightarrow w' \rightarrow \text{softmax}$
 $\downarrow$
 OHE

$\varepsilon_{w=h} =$ γ_1
 γ_2
 γ_3
 γ_4
 γ_5

uw_1 the

uw_2 cat

uw_3 sat

uw_4 on

uw_5 mat

$[1, 2]$

$[0, 1, 0, 0] \rightarrow$

TF-IDF $\rightarrow$

word 2 vec

The

$[x \ 1 \ 2] \rightarrow 3$

5 unique words $\rightarrow$ HL - calculation $\rightarrow$? $\rightarrow$ 4

Fast Text

Break the word into subwords

n-gram = 3

word = apple

< apple > $\rightarrow$ < ap, app, ppl, ple, le >
① ② ③ ④ ⑤

$\rightarrow$ word 2 vec

$\rightarrow$ w & w'

< ap	x_1	x_2	x_3
app			
!			
:			
le >			

$\rightarrow$ w matrix

all vocab

ϵ_{apple} = average of apple vocab only

$$h = \frac{1}{5} \sum_{i=1}^5 \epsilon_i$$

$h = [x_1 \ x_2 \ x_3] \rightarrow$ represent apple $\rightarrow$ not present in w

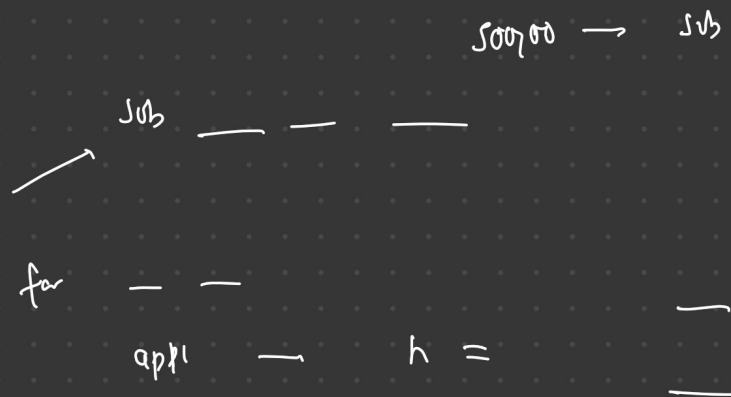
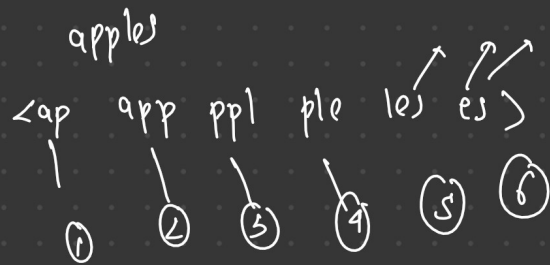
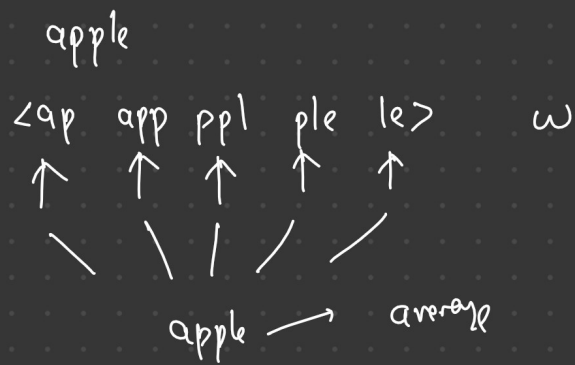
$$w' = v_w \times D$$

= word vocab size
 $\times$ Embedding

$$w' = v_{w'} \times D$$

= sub word vocab size
 $\times$ Embedding

apple $\rightarrow$ apple $\rightarrow$ w $\rightarrow$ < ap, app, ppl, ple
/ le, le >



GloVe embedding

uses word-occurrence matrix to learn embeddings.

(1) Build co-occurrence matrix.

vocab of 5 words → king, queen, Man, woman, Apple

→ count, how often a word appear together in a huge corpus.

	king	queen	Man	woman	Apple
king	10	3	8	2	0
queen	3	12	2	6	0
Man	8	2	15	5	1
woman	2	6	5	10	0
Apple	0	0	1	0	7

$x_{ij} = \log(x_{ij}) \rightarrow$ we take $\log$ of each non-zero value

word ↓	king	queen	Man	woman	Apple	← context
king	2.30	1.10	2.08	0.69	—	
queen	1.10	2.48	0.69	1.79	—	
Man	2.08	0.69	2.71	1.61	0	
woman	0.69	1.79	1.61	2.30	—	
Apple	—	—	0	—	1.95	

→ Define w & w' (both are trainable)

$w \rightarrow$ word embedding

$w' \rightarrow$ context embedding

assume $D=2$, i.e. Dimension of embedding.

	w - 2D vector
king	$w_{\text{king}} = [w_{k1}, w_{k2}]$
queen	$w_{\text{queen}} = [w_{q1}, w_{q2}]$
⋮	
Apple	$w_{\text{apple}} = [w_{a1}, w_{a2}]$

	w' - 2D vector
king	$w'_{\text{king}} = [w'_{k1}, w'_{k2}]$
queen	$w'_{\text{queen}} = [w'_{q1}, w'_{q2}]$
⋮	
Apple	$w'_{\text{apple}} = [w'_{a1}, w'_{a2}]$

→ Core objective function

$$J = \sum_{i,j} f(x_{ij}) (\underbrace{w_i^T w_j'}_{\text{dot product}} + \underbrace{b_i}_{\text{bias}} + \underbrace{b_j'}_{\text{bias}} - \log(x_{ij}))^2$$

$w_i^T w_j'$ → dot product of word embedding w & w'

b_i, b_j' → are bias term

$f(x_{ij})$ → is a weighting function to reduce the influence of rare words.

$$f(x_{ij}) = \begin{cases} \left(\frac{x_{ij}}{x_{\max}} \right)^\alpha, & \text{if } x_{ij} < x_{\max} \\ 1, & \text{otherwise} \end{cases}$$

$\alpha =$ A small constant, typically 0.75

$x_{\max}$ → Determine if word too frequent, threshold for co-occurrence.

If too high, set to 1.

if $x_{\max} = 10$

$$x_{ij} = x_{\text{king}, \text{king}} = 10 \quad f(x_{ij}) = 1 \quad \text{because } 10 < 10$$

$\uparrow \quad \uparrow$
 $i \quad j$

→ Calculation

king, queen

$$w_{\text{king}} \cdot w_{\text{queen}}' + b_{\text{king}} + b_{\text{queen}}' = \log(x_{\text{king}, \text{queen}})$$

$$[0.8, 1.2] [0.5, 0.7] + 0.3 + 0.2 = 1.10$$

$$(0.8 \times 0.5) + (1.2 \times 0.7) + 0.3 + 0.2 = 1.10$$

$$\underline{1.74} = \underline{1.10}$$

loss $\rightarrow$ BP $\rightarrow$ GD

finally, word embedding $= E = w + w'$

$$E_{king} = w_{king} + w'_{king}$$

- $\rightarrow$ Reduce the impact of extremely frequent word pair
- $\rightarrow$ prevent rare word from dominating the learning process.

Tokenization $\rightarrow$

process of taking input text & partitioning into small unit called tokens:—

words $\rightarrow$ "The", "A", "A", "jump", " "

subwords $\rightarrow$ "The", "jump", "##s", "lap"

character $\rightarrow$ "T", "H", "s", "."

$\rightarrow$ Text = "Hi, how are you?"

$\rightarrow$ Normalization $\rightarrow$ lower:

hi, how are you? $\rightarrow$

"Hi, how are you?" $\rightarrow$ white space $\rightarrow$ a.split()
 "Hi", "how", "are", "you"